

Java Notes

this keyword :

- Used to reference the current object in java.

Shadowing : When the input parameter shares the same name as that of the instance variable, the shadow of the input parameter falls on the instance variable and the instance variable remains unchanged, this concept is known as shadowing.

- Due to shadowing, *JVM* is unable to distinguish between variable and instance variable.
- To get rid of Shadowing, we use *this*.

Example :

```
class account{
    String name = "Bharath";
    void newName(String name){
        this.name = name; //this refers to reference variable rather than
parameter.
    }
}
```

Types of Constructors:

2 types of constructors :

- Default Constructor

```
public class BankAccount {
    int balance;
    BankAccount(){
        balance = 5000;
    }
}
//BankAccount Ram = new BankAccount();
```

- Parametric Constructor

```
public class BankAccount {
    int balance;
    BankAccount(int bal){
        balance = bal;
    }
}
```

```

}
//BankAccount Ram = new BankAccount(1000);

```

Can Object have multiple states at the time of its creation? If yes, how is it achieved?

Yes, an object can have multiple states while creation. It can be created by *Default Constructor* or *Parametric Constructor*. We can also have multiple constructors. See the example

```

public class BankAccount {
    String name;
    int balance;
    String bankName;
    public class InnerBankAccount {
        String innerName;
        int innerBalance;

        InnerBankAccount(String name, int balance) {
            this.innerName = name;
            this.innerBalance = balance;
        }

        void display() {
            System.out.println("Inner Account Name: " + innerName);
            System.out.println("Inner Account Balance: " + innerBalance);
        }
        protected void BankAccount1(String name) {
            this.innerName = name;
        }
    }

    void setBalance(int balance) {
        this.balance = balance;
    }
    BankAccount(String user, int bal, String bN){
        name = user;
        balance = bal;
        bankName = bN;
        System.out.println("User Created.");
    }
    BankAccount(int bal){
        balance = bal;
        System.out.println(balance);
    }
    BankAccount(String user){
        name = user;
        balance = 5000;
        System.out.println(balance);
    }
    BankAccount(){
        balance = 5000;
        System.out.println(balance+" Default");
    }
}

```

```
}

public void deposit(int amount) {
    balance += amount;
    System.out.println("Deposited: " + amount);
}
public void deposit(int amount, String method) {
    System.out.println(method);
    deposit(amount);
}
}
```

Constructor:

- A constructor is a **code block** that is used to initialize a object.
- It has the same name as the class in which it resides.
- It is syntactically same as a method. But it is **not** a method. Because we can not call a constructor again and again in a single lifetime of a object. We can call a method as many times as we want.
- Constructor is automatically invoked whenever an object of class is created. you cannot invoke a constructor.
- It is called an executed only once per object.
- Default Constructors are the ones which are created without parameters.

There are 2 types of constructors. They are

- Implicit Constructor
- Explicit Constructor

The *Implicit Constructor* only works when there is **no** constructor declared.

If we have one or more constructors defined, JVM will not invoke a *Implicit Constructor*. We have to declare a *Explicit Constructor*.

If there are two constructors with same number of parameters, JVM chooses a constructor only when

When we create a instance variable, it gets created for every object. But incase of methods they don't get copied like instance variables, they belong to the class. Similarly, if we want a variable to be same for all objects (be wrt to the class) then we make it a static variable.

Static Variable

A variable which belongs to the class is called as a static variable.

- **Perm gen** is a area which stores the *static variables*.
- **Heap** is where the objects are stored.

```
class Book{
    static int stock; // same for all objects.
```

```
String bookName;
double bookPrice;
Book(String name, double price){
    stock++; // Keeps track of how many books are there.
    bookName = name;
    bookPrice = price;
}
}
```

- We don't need to create an instance of the class (*a.k.a object*) to access a Static Variable.

```
System.out.println(Book.stock); // works.
Book book1 = new Book("name1", 2000)
System.out.println(book1.stock); // works here too.
book1.stock = 20; // Will change the original stock.
```

- Static Variables are created when the class is loaded and they die when the class is unloaded.
- Static Variables are created because of **Memory Efficiency**.

What is a Static Method?

To invoke a static method, we don't have to create an object, but whereas the regular methods need to be invoked only after creating an object. But both the methods aren't copied for every method.

Why is the *main* method Static?

JVM invokes main without creating the filename-object. It directly invokes `filename.main()`. If we made it just `public void main()` instead of `public static void main()` then the JVM will not be able to invoke main directly. This is the reason for it being static.

Deepa mam Answer:

Whenever a method is declared as static, it can be directly invoked without creating an object. Static methods belong to class. So, as soon as the class is loaded into JVM, the static method is available to be invoked.

Whenever you declare a method as static, instance variables cannot be used in this context. Why?

- Static variables can't use instance variables. They can, however, use local/static variables.
- This is also the reason for not being able to use non static methods in a static context.
- Memory for static members is allocated only once at class loading.
- Cannot use *this* or *super*

Can a reference variable be declared as Static? [Single ton designing pattern.]

Initializers

Read Initializers from the Internet.

- Run This to see more about initializers.

```
class Bank{
    int balance;
    static String BankName = "something 1st";
    Bank(int b){
        balance = b;
    }
    {
        BankName = "1";
        System.out.println("Initializer 1 Called "+BankName);
    }
    static {
        BankName = "something";
        System.out.println("Static Initializer Called "+BankName);
    }
    {
        BankName = "2";
        System.out.println("Initializer 2 Called "+BankName);
    }

    public static void main(String[] args){
        Bank b1 = new Bank(2);
        System.out.println(b1.balance);
    }
}
//Static Initializer Called something
//Initializer 1 Called 1
//Initializer 2 Called 2
//2
```

{ ... } is called **Instance Initializer**, its called by JVM automatically before anything else, and if there are multiple then they are called in the order they are defined.

Output :

```
Static Initializer Called something
Initializer 1 Called 1
Initializer 2 Called 2
2
```

Anything that is instance can use static but anything that is static cannot use instance.

Why do we need Initializers?

- **Static Initializer** : In order to initialize static variables or to execute any piece of code when a class is loaded, we use static initializers.
- **Instance Initializer** : If we write a code and we need that to be executed when a instance is created, we use Instance Initializer.
- Initializers are executed before constructors.

What is Constructor Chaining?

In a class, there can be several constructors present (overloaded constructors) and all these constructors should be invoked while creating a single object. Then we used the concept of constructor chaining.

We do this with `this(args);`

Why do we need constructor chains?

Depending on the data available, different constructors are invoked, resulting in different states of the object. But, if all the objects need to come together and have the same state, then we need to chain these constructors.

Order of execution

Lets see if this goes to 2 pages XD

1. Class loaded
2. Static Variables load
3. Static Initializers load
4. Static methods load
5. `Bank SBI = ...` when this happens its called a instance being created.
 - `... = new Bank(args)` is where object is constructed.
6. Instance Variables load
7. Instance Initializers load
8. Constructors called
9. methods load

Inheritance

When one class (A) attains the state and behavior of another class (B), it is said to be following inheritance property. A is the child, derived, specialized. B is the parent, base, generalized.

- Instead of having repetitive code in the similar classes, we can save the common functionality in a common class.
- Changes made in the base class is affecting all the child classes.

See Example

```
public class person {  
    String name;
```

```

    int age;
    static int i = 10;
    void eat(){
        System.out.println(name+" is eating.");
    }
    public static void main(String[] args) {
        Teacher deepa = new Teacher("Deepa",40);
        deepa.eat();
        deepa.work();
        System.out.println(Teacher.i); // ! Static Variables are inherited.
    }

    {
        System.out.println("Person Initializer called");
    }
    person(String name, int age){
        this.name = name;
        this.age = age;
        System.out.println("Person Constructor called");
    }
}

class Teacher extends person{
    {
        System.out.println("Teacher Initializer called");
    }
    Teacher(String name, int age){
        // super(name, age); // * this is fine, but see below.
        super("smthing",777);
        this.name = name;
        this.age = age; // ! This works too because "this" changes the name and
age set by super because its called afterwards.
        System.out.println("Teacher Constructor called");
    }
    void work(){
        System.out.println(this.name+" is teaching.");
    }
}

class Student extends person {
    double GPA;
    // * Required to have a constructor with super defined.
    Student() {
        super("someone", 18);
    }
}

/*
/*
Pracice Program

class Person {
    String name;
    int age;

```

```

    String email;
    void eat(){
        System.out.println(this.name+" is eating.");
    }
    void Display(){
        System.out.println("Name :"+name);
        System.out.println("Age :"+age);
    }

    Person(String name,int age){
        this.name = name;
        this.age = age;
    }
}

class Teacher extends Person{
    {
        System.out.println("Teacher Initializer called");
    }
    float exp;
    Teacher(String name,int age){
        super(name,age);
        // this.name = name;
        // this.age = age;
        System.out.println("Teacher Constructor called");
    }
    void work(){
        System.out.println(this.name+" is teaching.");
    }
}

class Student extends Person {
    double GPA;
    void study() {
        System.out.println("Studying lessons");
    }
}

class Staff extends Person {
    double salary;
    void maintain() {
        System.out.println("Maintaining school facilities");
    }
}
*/

```

- A parent class CAN'T access the state and behavior of child, but a Child CAN access all the state and behavior of the parent.

Whenever a child class object is getting created, the base class object gets created prior to it.

- The objects are created like this :

- when you say `deepa.name` it checks if `Teacher` has name, then if it doesn't then it checks `person` for the name.
- When it finds the name it does not create a copy of instance variable for `Teacher`, **it creates instance variable name in the instance of the person.**

Static variables are inherited to children.

`super()` :

- `super()` is the reference to the parent constructor.
- `this()` is the reference to the current instance variable.
- It should **always** be in the first line of the constructor.

Create a librarian and a principal. Each class should have its own state and behavior. Let the classes have only parameterized constructors. principle and librarian and teacher eat their lunch together after performing their duties.

What is overriding?

When the child class / derived class requires the same functionality as the parent class but wants to change its implementation, then it wants to re define that method in its own class. This process is called as over riding.

There are several rules for over riding :

1. The name of the method should be same.
2. Parameter list (no. of parameters, data type, and order of parameters) should be same.
3. Return type should be same / should be the subclass of the return type of the overridden method. (Law of Covariance)
4. Access modifiers should not be restricted.
5. Final Method cannot be over ridden.

The one in parent class is over ridden, the one in child class is over riding.

`final` keyword :

It's basically like `const` of python. All final objects are also usually declared as static.

We can use this keyword with :

Instance Variable

You can only assign the value in one of 2 places, they are :

1. While declaring it

```
class Person{
    final String name = "Bharath";
}
```

2. In the constructor

```
class Person{  
    final String name;  
    Person(){  
        name = "Bharath";  
    }  
}
```

Local Variable

We can assign it while declaring, or do this :

```
final String name;  
name = "Bharath"; //immediately in next line
```

Reference Variable

We can not change the reference variable's value (Memory address), but we can use the reference variable to change what it's referring to.

```
final Person p;  
p = new Person();  
  
p.name = "Bharath"; // works fine
```

Methods

Overriding can not be done on methods that have been declared as final. We can not change the methods once they are declared as final.

- But we can inherit these methods.

Others

- Initializers and constructors can not be final, because there can be more than one of constructors and initializers. And we can not say that they wont be changed.
- Whenever the class is declared as final, we can not inherit the class anymore. Inheritance will get disabled.

Abstract

Whenever a method is declared as abstract, it doesn't have any body.

Why is it necessary for the subclass to override abstract methods?

When we declare a method as abstract, we are enforcing that all the children have that method. Even if we don't give it a implementation/ a body, we know that children will have to have that method and they will implement the functionality by adding their own body.

- When all the children of a class override a certain method, we should delete the body of that method and declare it as abstract.
- When the parent wants to enforce a method, but don't know how the children will use that method, it can define that method as abstract.
- If we don't want people to create objects of a given class, we can make it abstract.
- We have to

Whenever a method is declared as abstract in a class, class should be declared as abstract too.

Abstract classes are made because :

- all methods abstract
- implementing it because a method is abstract
- don't want to have objects.

why is a class abstract when a method is abstract?

If we create a object from the class, if someone calls the abstract method, that method will not have a body because its abstract, so it will not know its implementation.

Why methods and classes can't be both abstract and final?

Final stops inheritance, abstract demands inheritance.

Car Plan

car has:

- Accelerate
- Brake
- gears
- engine
- steering
- speedometer

Automatic

- all from car
- Sports model
-
-

Manual

- all from car
- clutch
- manual gears
-

```
public class Cars {  
  
}  
  
abstract class Car {  
    void accelerate(){  
  
    }  
    void brake(){  
  
    }  
    abstract void gears();  
    abstract void engine();  
    abstract void steering();  
}  
  
class Manual extends Car {  
    void clutch(){  
  
    }  
    void engine() {  
  
    }  
    void gears() {  
  
    }  
    void steering(){  
  
    }  
}  
  
class Auto extends Car {  
    void sportsModel(){  
  
    }  
    void engine() {  
  
    }  
    void gears() {  
  
    }  
    void steering(){  
  
    }  
}
```

and

> ## Go through [these questions](./questions/10%20questions/)

- An **abstract** class can have a mixture of both concrete methods and **abstract methods**(No body).
- The JVM tries to protect a disaster, which is a method we invoked on an **abstract** class. If the method which is **abstract**, where will the JVM search for the body? So it does not allow to invoke methods on a **abstract** class.

Packages

We define a class / file to be in a **package** by writing

```
```java
package Kmit;
class example {
 //code in example now belongs to package kmit
}
```

- To compile a class inside a package, we do **javac classname.java -d .** where **-d** stands for directory and **.** stands for current dir.
- When we run that command then we get a folder with name "kmit".
- All packages can access **public** things from anywhere.
- We run **classname.class** by running **java kmit.classname**

## Single ton design pattern

There is only one object in the whole system.

```
public class single {
 int i;
 public static single a;
 static int counter;
 private single(int i){
 this.i = i;
 counter++;
 }

 public static single create(int i){
 if (counter < 2){
 return a = new single(i);
 }
 else {
 System.out.println("no obj created.");
 return a;
 }
 }
}
```

```
 }
 void display(){
 System.out.println(a);
 }
}
```

## Polymorphism

It is of 2 types :

- Runtime Polymorphism / Dynamic Polymorphism
- Compile time Polymorphism / Static Polymorphism

## Inheritance

Inheritance

It is of 2 types :

- Single inheritance
- Multi level inheritance

**Java only supports *single inheritance*.**

## Interfaces

An Interface is a contract that specifies what methods a class should implement, without specifying how these methods should be implemented.

Interfaces provides a way to achieve abstraction and multiple inheritance in java, as a class can implement multiple interfaces.

**Coding Convention:** Define the name of interface as **IPlayer** or **IPerson**, etc... basically add **I** to the start of the name.

```
interface IPlayer {
 void win();
 void lose();
 void play();
}

interface IPerson {
 void eat();
 void sleep();
}

class chessPlayer implements IPlayer, IPerson {
 public void win() {...}
 public void lose() {...}
 public void play() {...}
```

```

 public void eat() {...}
 public void sleep() {...}
}

```

Basically same as **Abstract Class** but we can implement multiple Interfaces, so its possible to implement both IPlayer and IPerson.

## API

API - Application Programming **Interface**

An interface is a public contract between an Implementing platform and the user.

So they publish a interface with **abstract methods**. Also there is a **.jar** file which has implementation classes in binary format. Hence we can use their code with APIs using these 2 things.

There are 3rd party libraries which provide us with implementation classes. These libraries are the binary files.

**Abstraction** : Hiding away the implementation of the functionality and giving the users a way to use their methods through APIs.

- By default, all the methods in the Interface are **abstract** and **public**. We can change this and give a body to a method of interface by using a keyword **default**

```

interface ITest{
 void tested(); // abstract and public by default
 default void testing() {
 System.out.println("This works fine."); // this can have body because we
 used keyword default.
 }
}

```

- All variables are static and **final** by **default**.
- Interface references can be used to point the objects that implements their associated interface.

Interface can't implement another interface.

- We can have empty interfaces, they are called **Marker Interfaces**.

Functional Interface :

- A functional interface is a interface with only single abstract method. They can have any number of static or default methods. These are used to create **lambda functions** in Java.

```

@FunctionalInterface
interface kmit {
 void m();
}

```

```
class testFI {
 public static void main(String[] args){
 kmit i = ()->{
 //function
 System.out.println("Lambda function")
 }
 i.m(); // prints Lambda function
 }
}
```

See [this](#) file.

## What do you mean by loose coupling of code?

Whenever you create an application, you create 'N' no. of classes, these classes can have "is-a" relationship or "has-a" relationship or relationship with interfaces. The code should be written in such a way that any addition/deletion in a problem statement should not affect the code. Nothing should be re-written.

What is an Exception: An exception in Java is an object that represents an error or an unexpected event that occurs during program execution (runtime).

When such an event happens, Java creates an object of a particular Exception class, and this object contains:

- i. The type of exception
- ii. The description (message) of the error
- iii. The stack trace (the list of methods that were running at the time)

Then this object is "thrown" to the Java Runtime System.

How Exceptions Happen: The Following steps, when an exception occurs:

Step 1: Error Detected at Runtime When the JVM executes your program, it continuously monitors for abnormal conditions (errors).

Example: `int x = 10 / 0;`

When JVM tries to divide 10 by 0, it detects an illegal arithmetic operation.

Step 2: JVM Creates an Exception Object

At that moment, the JVM creates an object representing the error. This object belongs to a subclass of the Throwable class.

For the above example:

`ArithmeticException ex = new ArithmeticException("/ by zero");`

Step 3: The Exception is Thrown

The exception object is thrown to the JVM using the throw mechanism — automatically by JVM or manually by programmer.

a) If JVM throws it automatically then it is built-in exception. b) If programmer throws it manually then it is user-defined or manually thrown exception or custom exception.

#### Step 4: JVM Looks for a Matching catch Block

Now, the JVM starts searching for a matching catch block that can handle this exception.

It looks in this order: The current method (try-catch inside method). If not found → the method that called this method Continues going up the call stack.

If no method handles it → JVM terminates the program

This process is called Exception Propagation.

#### Step 5: Control Transfers to the Catch Block

If a matching catch block is found, control is immediately transferred there, and the rest of the code in the try block is skipped.

Example:

```
try { int x = 10 / 0; // Exception occurs here System.out.println("This will not execute"); } catch
(ArithmeticException e) { System.out.println("Exception caught: " + e); } System.out.println("Program
continues...");
```

Output:

Exception caught: java.lang.ArithmeticException: / by zero Program continues...

#### Step 6: If No Catch Block Found

If no matching catch block is found in the entire call chain:

JVM prints the exception name, description, and stack trace.

Program terminates abnormally.

Example:

```
int a = 10 / 0; System.out.println("Unreachable code");
```

Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero at MyProgram.main(MyProgram.java:3)

Exception Propagation Example class Test { static void divide() { int x = 10 / 0; // exception occurs }

```
static void compute() {
 divide(); // called by compute()
}
```

```
public static void main(String[] args) {
 try {
 compute(); // exception propagated here
 } catch (ArithmeticException e) {
 System.out.println("Handled in main(): " + e);
 }
}
```

```
}
```

Explanation:

Exception occurs in divide().

No catch block → sent to compute().

No catch block there → sent to main().

main() has a catch block → handled.

Output:

Handled in main(): java.lang.ArithmeticException: / by zero

Types of Exception Generation: i. Built-in: occurs when When Java code violates rules, Throws It by JVM, eg: Division by zero, invalid index ii. User-defined Exception: occurs when When programmer defines own error condition, Throws It by Programmer, eg: Age < 18, invalid marks, etc.

Exception Lifecycle Error detected → Exception object created → Exception thrown ↓ JVM searches for handler  
↓ If found → catch block executes If not found → program terminates